GPS Solutions (2025) 29:167
https://doi.org/10.1007/s10291-025-01909-6

RESEARCH

Assessing the robustness of machine learning strategy for GNSS/INS vehicle positioning solutions enhancement

Wenzong Gao[1] · **Yanming Feng**[1]

Received: 23 May 2024 / Accepted: 11 June 2025 / Published online: 11 July 2025

Abstract
Accurate and reliable vehicle positioning during extended Global Navigation Satellite System/Inertial Navigation System outages remains challenging due to noise, outliers, and biases in input data. This study evaluates the robustness of machine learning (ML) models under two specific conditions: (1) noise and outliers present in training target data, and (2) errors and biases within input data. Two ML methods–Long Short-Term Memory (LSTM) and Gradient Boosting Decision Tree (GBDT)–were evaluated using the proposed methodology. Results indicate that both ML methods significantly enhance horizontal positioning accuracy compared to the conventional Kalman filter (KF). Specifically, LSTM reduces horizontal RMSE by approximately 85.8% (from 39.5 to 5.6 m), while GBDT achieves a reduction of approximately 77.7% (to 8.8 m). Additionally, the robustness analysis confirms that the proposed LSTM-based strategy demonstrates substantial robustness by maintaining consistent prediction accuracy, even in the presence of significant errors and outliers in training data, as well as being minimally impacted by errors and biases introduced in input variables.

Keywords GNSS · INS · Machine learning · Robustness · Long short-term memory · Gradient boosting decision tree

Introduction

This study evaluates the performance of two machine learning (ML) methods—long short-term memory (LSTM) and gradient boosting decision tree (GBDT)—on horizontal position prediction for global navigation satellite system/inertial navigation system (GNSS/INS) integration system during GNSS signal outages. More importantly, the robustness of the proposed ML strategy was studied, including (1) the study on impact of outliers and noise in response data on trained models, and (2) impact of biases and noise in input data on prediction accuracy.

The development of GNSS has significantly enhanced navigation technologies, offering crucial positioning and timing services for a range of applications. GNSS offers benefits such as global coverage and consistent performance (Yigit et al. 2014). However, it also encounters challenges, including vulnerability to signal outages caused by obstructions like buildings and tunnels. Additionally, GNSS solutions may include outliers due to incorrect integer fixes, especially in poor observational environments, and can display high-level noise when using techniques like differential GPS (DGPS). Moreover, GNSS typically provides lower update frequencies, which may not suffice for applications that demand real-time, high-frequency positioning (Al Bitar et al. 2020). These limitations can compromise the accuracy and reliability of essential navigation systems.

In contrast, the INS offers high-frequency updates and operates independently, not relying on external signals (Amami 2022). An INS is comprised of two primary components: the inertial measurement unit (IMU) and a navigation computer. The IMU is equipped with a triad of gyroscopes and accelerometers, which are arranged orthogonally to measure angular velocity and linear acceleration, respectively. These measurements are fundamental to deriving the navigation parameters, specifically position, velocity, and attitude. To transform the raw inertial data from the IMU into these navigational outputs, a process known as mechanisation is employed (Yang et al. 2014). This involves the application of kinematic equations, which are executed by the navigation computer to accurately compute and integrate

✉ Wenzong Gao
wenzong.gao@hdr.qut.edu.au

Yanming Feng
y.feng@qut.edu.au

[1] Faculty of Science, Queensland University of Technology, Brisbane 4000, Australia

the navigation data (Zhao et al. 2018). While INS is beneficial for short-term navigation due to its high update rate, it is prone to errors in velocity, attitude, and position solutions that accumulate over time, known as drift, leading to increasing inaccuracies if not periodically corrected. Furthermore, INS cannot determine absolute positions on its own. It can only calculate the vehicle's relative position from a known starting point by accounting for distance travelled and orientation (Goodall 2009).

In order to address the limitations of independent GNSS or INS operations and to leverage the strengths of both systems, they are commonly integrated into a single system known as GNSS/INS. Depending on the degree of integration, GNSS and INS can be integrated in three main ways: loosely, tightly, and deeply coupled (Schmidt et al. 2011). In loosely coupled integration (LCI), the GNSS solution is merged with IMU data to produce the final output. In tightly coupled integration (TCI), however, raw GNSS measurements are also incorporated into the integration process. Deeply coupled integration (DCI) is the most complex form, involving significant interdependence between GNSS and INS. This integration combines GNSS solutions and raw measurements from both GNSS and IMU, while also providing IMU feedback to improve GNSS tracking and positioning algorithms. As a result, DCI can considerably reduce outliers and noise in position solutions compared to standalone GNSS solutions.

The primary algorithms for integrating GNSS and INS data include the Kalman filter (KF) and its variants, such as the extended Kalman filter (EKF) and the unscented Kalman filter (UKF) (Hu et al. 2018; Meng et al. 2016). These filters are computationally efficient, making them ideal for real-time applications. KF provides accurate geo-referencing solutions, assuming a continuous GNSS signal for precise dynamic and stochastic error modelling of the systems. During GNSS outages, KF operates in prediction mode to correct INS measurements based on the system's error model (Al Bitar and Gavrilov 2021). This method is particularly effective in high-grade systems with well-characterised stochastic errors but becomes more complex with low-end MEMS-based IMUs due to their inherent noise and bias instability (Hong et al. 2005). Additionally, as a standard processing method, the KF struggles to handle complicated dynamic models with multiple inputs and cannot effectively utilise long periods of data. While it can make accurate predictions over short periods, significant errors arise over longer periods (Dasanayaka and Feng 2022).

Contrary to KF methods, which depend on predefined mathematical models and detailed prior knowledge, ML methods are inherently more adaptable, learning directly from data without the need for explicit programming. ML's empirical and adaptive approach enables it to manage complex, nonlinear patterns across different systems without the need for redesign or tuning specific to various sensors or environments. This flexibility sharply contrasts to the sensor-specific and linear nature of KF, emphasising ML's broader applicability and ease of integration into diverse applications, although it requires substantial data for effective training (El-Sheimy et al. 2006).

Numerous studies have explored the application of ML techniques in GNSS/INS integration. Some studies combined ML with KF to enhance overall navigation accuracy by leveraging the strengths of both ML and KF. Predominantly, these studies have employed various ML approaches, achieving satisfactory results in position prediction. These include the multilayer perceptron neural network (MLPNN) (Sharaf et al. 2005; Yao et al. 2017), adaptive neuro-fuzzy inference system (ANFIS) (Abdel-Hamid et al. 2007; Noureldin et al. 2009), radial basis function neural network (RBFNN) (Wang et al. 2007; Chen and Fang 2014; Wang et al. 2015), wavelet neural network (WNN) (Chen et al. 2013), extreme learning machine (ELM) (Jingsen et al. 2016), back propagation neural network (BPNN) (Wang et al. 2019), LSTM (Fang et al. 2020), and nonlinear autoregressive neural networks with external inputs (NARX) (Al Bitar and Gavrilov 2021; El Sabbagh et al. 2023).

Several studies have adopted ML for GNSS/INS integration as an alternative to KF. El-Sheimy et al. (2006) used position update architectures (PUA) to integrate GNSS with IMU data, training an MLPNN to maintain positioning accuracy during GNSS outages. Kw et al. (2008) suggested replacing PUA's MLP with a cascade-correlation network (CCN) for its adaptive capabilities in dynamic environments, while Adusumilli et al. (2013) proposed substituting it with a random forest regression (RFR) algorithm for its robust handling of complex relationships. Additionally, the P–δP approach, first introduced by Noureldin et al. (2003), functions as a model-less module that mimics KF functionality by estimating INS position errors. Sharaf et al. (2005) recommended using an RBFNN in the P–δP architecture for its simpler, faster-training architecture which surpasses KF in accuracy. Noureldin et al. (2007) enhanced this approach with a temporal window-based cross-validation method to fine-tune ANFIS parameter updates, further improving GNSS/INS integration accuracy.

The body of research surrounding the application of ML in the integration of GNSS and INS has indeed showcased ML's potential for enhancing navigation solutions. However, several gaps remain, warranting attention for the refinement and future direction of such technologies. (1) Robustness concerns: The research on ML models' prediction accuracy is thorough, yet there is a lack of focus on their robustness–defined as the model's consistent performance when faced with new, unseen data variations. This gap is particularly concerning for safety critical applications of vehicle navigations. (2) Use of alternative vehicle data

in ML processing: It has been shown that it's possible and beneficial to merge supplementary datasets like the vehicle's On-Board Diagnostics II (OBD-II) speed data into the ML framework without necessitating new hardware. This highlights the potential for ML models to become more practice by utilizing readily available vehicle sensor data, thus enhancing the richness of the input without escalating costs. (3) ML computational intensity concerns: While the incorporation of a multitude of input features, such as raw IMU data and derived parameters, can augment the model's insight, it also substantially increases the computation and training duration. This situation poses a practical challenge for time-critical operations. An ideal model needs to balance input complexity with timely output. (4) Neural network (NN) drawbacks: Although NN-based methods are prevalent and potent in handling time-series data, their computational demands are significant. The extensive training time and the complexity of NN models can encumber real-time vehicle navigation operations, and (5) Accumulative error risks: Some approaches predict incremental positional updates and compile them to estimate the current states (El-Sheimy et al. 2006). While computationally efficient, they are prone to the accumulation of prediction errors. A singular mistake in estimating a positional change can cascade, causing the system to drift increasingly away from the true position over time. Overall, the current state of research underlines the importance of developing robust, efficient, and scalable ML solutions that can meet the reliability demands of real-world navigation and avoid the pitfalls of error propagation, computational delays, and limited generalizability.

This study will address the above limitations, focus on examining the robustness of the proposed ML strategy for integrating GNSS/INS systems. Though the robustness of ML model has been studied by some researchers, its concept in those studies remains ambiguously defined. Previous research has introduced several specific notions of robustness, including adversarial robustness (Dong et al. 2020), robustness to distribution shifts (Hendrycks et al. 2021; Koh et al. 2021), and shortcut learning (Geirhos et al. 2020). In addition, the survey by Frénay and Verleysen (2013) discusses various methods for managing label noise in classification tasks, where robustness is considered a measure of an algorithm's ability to resist performance degradation due to inaccuracies in the target labels. In this study, we discuss robustness from two perspectives. The first perspective is akin to the definition of robustness to distribution shifts, particularly covariate shift, where the distribution of inputs in the testing data differs from that in the training data, yet the relationship between inputs and outputs remains unchanged (Sugiyama et al. 2007). Specifically, we aim to determine if the model can still make accurate position predictions when the testing environment differs from the training environment, such as when a lower-level IMU is used to collect the test data. The second type of robustness we address is similar to that described in Frénay and Verleysen (2013), but with a focus on regression problems. We examine how outliers and noise in the response data of training sets, similar to label noise in classification, affect the performance of trained regression models. This is because there are usually outliers and noise in the response data, such as impact of wrong phase integers in RTK state solutions and large noises in DGPS state solutions. Therefore, this study will examine the performance statistics of ML models when trained on different response data, for example, data derived from DCI, RTK, and DGPS processing schemes.

In this study, we introduce the GBDT method for more efficient ML processing. Moreover, the position components are used as the outputs in the ML processing, rather than changes in position, to avoid the risks of error accumulations. Accordingly, the input variables were selected as elapsed time, IMU-derived position, OBD-II-derived position, and azimuth. Based on these settings, we can study the robustness of the ML strategy in processing GNSS/INS data. Specifically, this study seeks answers to four research questions in the ML pipeline for GNSS/INS data processing: (1) How to form the outputs variables in the ML process to avoid the impact of error accumulation in position increments; (2) How does the performance of ML in predicting positions vary during various outages; (3) How do outliers and large noise in the response data of training datasets affect the performance of trained models; (4) How do biases and noise in the input data affect the model's prediction accuracy.

The remainder of this paper is structured as follows: Sec. 2 delineates the methodology, encompassing the underlying principles of both the GBDT and LSTM methods. Sec. 3 introduces the data collection, format and preprocessing. In Sec. 4, we present an in-depth analysis of the performance of the ML methods in position prediction. Finally, Sec. 5 concludes the study, summarising key findings and implications.

Methodology

This section will first provide a framework for our ML strategy. The subsequent subsection will introduce the fundamental principles of supervised ML, focusing on how to select inputs and outputs for the ML process. Additionally, the architectures of LSTM neural networks and GBDT will be described, along with detailed explanations of the data collection process and data analysis methods.

Research workflow

This subsection provides an overview of ML processing, as illustrated in Fig. 1. The proposed ML strategy consists of a two-phase process: a training mode and a prediction mode.

In the training mode, when GNSS signals are available, the system integrates both GNSS and IMU data.

This includes GNSS raw measurements (carrier phase measurements ϕ and pseudoranges ρ), IMU raw measurements (changes in orientation $\Delta\Theta$ and changes in velocity Δv), and GNSS-derived position (P_G) and velocity (V_G). These are combined using an EFK within NovAtel's synchronised position and attitude navigation (SPAN) technology to produce refined estimates of position P_r, velocity V_r, and altitude A_r. As a DCI technology, SPAN enhances the INS by integrating GNSS data, which, in turn, significantly improves GNSS signal reacquisition, and reduces positioning errors such as outliers, thereby benefiting the ML training process (NovAtel Inc. 2022). The positions in N (N_r) and E (E_r) directions are handled independently in the processing. The SPAN-derived positions serve as the output for ML training and as ground truth to evaluate the ML prediction accuracy during GNSS signal outages. Four variables obtained from the INS and OBD-II systems were selected as inputs. The ML model, which defines the relationship between these inputs and the output, will be trained using the training datasets. The reason for choosing these specific variables as inputs and output will be discussed in the next subsection.

In prediction mode, when GNSS signals are unavailable, the INS and the vehicle's OBD-II system continue to generate the four input variables. These variables are then used to predict positions using the previously trained ML model.

Principle of supervised ML and definition of input and output variables

In supervised ML problems, models are trained using labelled samples known as training data. These samples are assumed to be independent and identically distributed, represented as:

$$D_{trn} = (x_1, y_1), (x_2, y_2), \ldots, (x_T, y_T) \tag{1}$$

where T denotes the number of samples in the training dataset. The input vector $x_t \in \mathbf{R}^n$ for $t = 1, 2, \ldots, T$, comprises one or more features. The goal of ML is to establish a mapping $\hat{Y} = \hat{f}(\mathbf{X})$ between $\mathbf{X} = (x_1, x_2, \ldots, x_T)$ and $Y = (y_1, y_2, \ldots, y_T)$, where $\hat{Y}$ represents the predicted output. The prediction process involves generating the output $\hat{y}_{T+1}$ for a new input vector x_{T+1} using the function $\hat{y}_{T+1} = \hat{f}(x_{T+1})$, once the relationship between $\mathbf{X}$ and Y is established.

In this study, we directly select the position as the output y_t for prediction. As discussed in Sect. 1, predicting the position directly, as opposed to predicting changes in position, can help avoid the accumulation of errors. The position results used for training are denoted as $N_r(t)$ and $E_r(t)$ for N and E directions, respectively.

After determining the output, it is essential to select input variables that effectively map to this output. Some studies have used only the INS position (Noureldin et al. 2009;

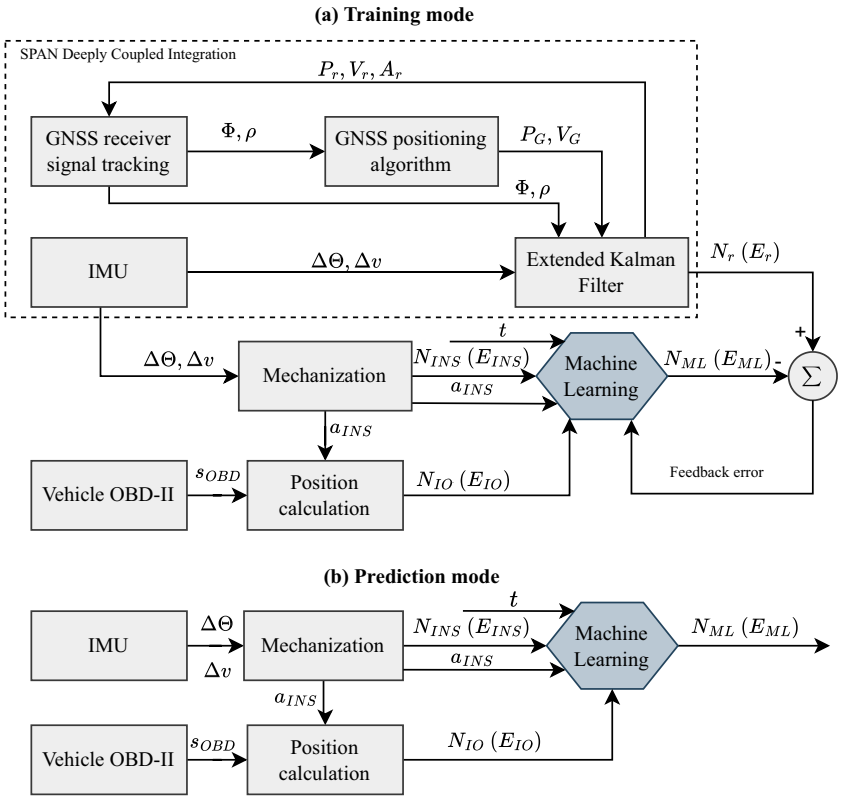

Fig. 1 Workflow for the proposed ML-based position prediction approach: **a** training mode; **b** prediction mode

Sharaf and Noureldin 2007), while others included a wider range of variables like raw IMU measurements and INS-derived solutions (Abdel-Hamid et al. 2007; Yao et al. 2017; Al Bitar and Gavrilov 2021). However, using too few inputs might not sufficiently capture the model's features, and too many can reduce ML process efficiency. It is advisable to exclude variables with weak correlations to the output, such as roll and pitch angles, which minimally affect northern and eastern coordinates. Overall, input variables should strongly correlate with the output for effective ML training, and it's beneficial to limit them to the necessary minimum for efficiency. Guided by these principles, this study utilised four variables as inputs: the elapsed time since the beginning of the outage, denoted by t; the cosine (or sine) of the azimuth, represented as $\cos(a_{INS})$ (or $\sin(a_{INS})$); the N (or E) coordinates derived from the INS, indicated as N_{INS} (or E_{INS}); and the N (or E) coordinates calculated using the azimuth and OBD-II-derived speeds, as outlined in Eq. 2, and denoted as N_{IO} (or E_{IO}). It is important to note that the azimuth was not used directly as an input variable due to its discontinuity. However, for simplicity, we sometimes refer to the use of azimuth as one of the inputs.

For the LSTM network with time-step of 1 or GBDT method, the input vector x_t can be formulated as:

$$x_t = \begin{cases} [t, N_{INS}(t), N_{IO}(t), \cos(a_{INS}(t))]^T \text{(for north)} \\ [t, E_{INS}(t), E_{IO}(t), \sin(a_{INS}(t))]^T \text{(for east)} \end{cases} \tag{2}$$

where

$$\begin{aligned} N_{IO}(t) &= \sum_{1}^{t} [s_{OBD}(t) \cdot \cos(a_{INS}(t))], \\ E_{IO}(t) &= \sum_{1}^{t} [s_{OBD}(t) \cdot \sin(a_{INS}(t))]. \end{aligned} \tag{3}$$

Before ML training, the input vector x_t is normalised using the z-score method, calculated as:

$$x'_t = \frac{x_t - \mu}{\sigma} \tag{4}$$

where μ and σ are the mean and standard deviation of x_t. This normalisation helps ML algorithms function efficiently and converge faster. A common challenge in ML modelling is overfitting, where a model fits training data well but performs poorly on new data. To prevent this, part of the dataset is reserved as a "validation dataset". This study employs LSTM and GBDT methods, which will be elaborated in the next subsection.

Long short-term memory

The LSTM network, a variant of the artificial recurrent neural network (RNN) architecture. However, unlike a classical RNN with a single hidden layer, where the input dataset $\mathbf{X} = (x_1, x_2, \ldots, x_T)$ processes the hidden vector sequences $H = (h_1, h_2, \ldots, h_T)$ and the output sequence $\hat{Y} = (\hat{y_1}, \hat{y_2}, \ldots, \hat{y_T})$ iteratively from $t = 1$ to T, the LSTM introduces a complex structure to overcome certain limitations:

$$h_t = \mathscr{A}\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right) \tag{5}$$

$$\hat{y}_t = \hat{f}_{LSTM}(x_t) = W_{hy}h_t + b_y \tag{6}$$

here $\mathscr{A}$ typically denotes the sigmoid function in conventional RNNs, with W_{xh}, W_{hh}, and W_{hy} representing weight matrices and b_h and b_y as bias vectors.

Although theoretically capable of generating complex sequences, traditional RNNs often encounter vanishing or exploding gradients, which limit their practical applications. To address these issues, the LSTM network incorporates a memory block within its hidden layer that includes a memory cell and three regulatory gates (input, forget, and output gates). This configuration allows the cell to maintain values over intervals and control the information flow effectively, retaining relevant information while discarding the irrelevant.

In LSTMs, the hidden vector h_t is calculated differently than in conventional RNNs:

$$i_t = \sigma\left(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i\right) \tag{7}$$

$$f_t = \sigma\left(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f\right) \tag{8}$$

$$c_t = f_t c_{t-1} + i_t \tanh\left(W_{xc}x_t + W_{hc}h_{t-1} + b_c\right) \tag{9}$$

$$o_t = \sigma\left(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o\right) \tag{10}$$

$$h_t = o_t \tanh\left(c_t\right) \tag{11}$$

where σ is the logistic sigmoid function, and i_t, f_t, c_t, and o_t correspond to the input gate, forget gate, cell activation vectors, and output gate, respectively. The W symbols denote the weight matrices, and b_i, b_f, b_o, and b_c are the biases for each gate.

Using these equations, the output $\hat{y}_t$ is computed as detailed in Eq. 6, with the entire output sequence $\hat{Y} = (\hat{y_1}, \hat{y_2}, \ldots, \hat{y_T})$ derived by iterating through the equations from $t = 1$ to T.

The architecture of the LSTM neural network, illustrated in Fig. 2, showcases how it is designed to circumvent the constraints of traditional RNNs by effectively managing information flow through its memory cell and gates. The next subsection will outline the workflow of this research

and describe the integration of the LSTM model into this process.

Gradient boosting decision tree

GBDT is an ensemble learning method that employs the concept of boosting to transform weak base learners into a strong collective model. The algorithm, developed by Friedman (2001), often utilises the classification and regression tree (CART) technique for its base learners. The ensemble model in GBDT is formulated as:

$$f_M(x) = \sum_{m=1}^{M} R\left(x;\Theta_m\right), \tag{12}$$

where $R\left(x;\Theta_m\right)$ is the m-th decision tree with parameters Θ_m, and M is the total number of trees.

The GBDT model is constructed iteratively:

$$\begin{aligned} &f_0(x) = 0, \\ &f_m(x) = f_{m-1}(x) + R\left(x;\Theta_m\right), \quad \forall m \in \{1,2,\ldots,M\}, \\ &f_M(x) = \sum_{m=1}^{M} R\left(x;\Theta_m\right). \end{aligned} \tag{13}$$

During the m-th iteration, the parameters $\hat{\Theta}_m$ are optimised by minimising the loss function:

$$\hat{\Theta}_m = \arg\min_{\Theta_m} \sum_{i=1}^{N} L\left(y_i, f_{m-1}\left(x_i\right) + R\left(x_i;\Theta_m\right)\right), \tag{14}$$

with $L(y,f(x))$ often chosen as the mean squared error (MSE), defined as:

$$L\left[y,f_m(x)\right] = \frac{1}{2}\left[y - f_m(x)\right]^2 = \frac{1}{2}\left[r - R\left(x;\Theta_m\right)\right]^2, \tag{15}$$

where $r = y - f_{m-1}(x)$ is the residual from the previous model f_{m-1}.

GBDT tackles the optimisation problem by fitting each new base learner to the negative gradient of the loss function with respect to the current model $f_{m-1}(x)$:

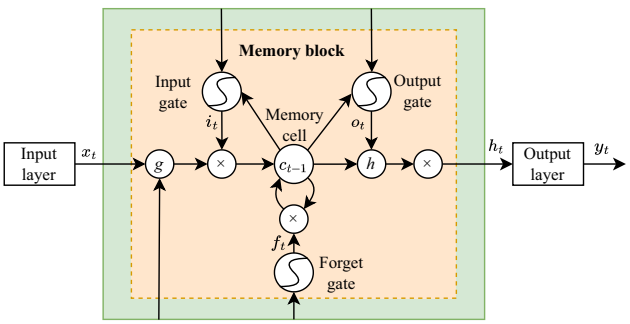

Fig. 2 LSTM neural network architecture (Ma et al. 2015)

$$\tilde{y} = -\left[\frac{\partial L[y,f(x)]}{\partial f(x)}\right]_{f(x)=f_{m-1}(x)}, \tag{16}$$

where $\tilde{y}$ represents the pseudo-residuals, an estimate of the true residuals.

Through this method, GBDT iteratively corrects the errors of its base learners, adapting to complex patterns in the data and building a forward stage-wise model that is both accurate and generalisable.

Evaluation metrics

To comprehensively evaluate the position prediction performance, six statistical metrics are used: root mean square error (RMSE), mean absolute error (MAE), scatter index (SI), correlation coefficient (CC), uncertainty interval (U95), and reliability percentage (Ebrahimi et al. 2024). The introductions of these metrics are as follows:

- RMSE: Measures the square root of the average squared differences between predicted values and true values. Lower RMSE indicates higher prediction accuracy.
- MAE: Represents the average of the absolute differences between predicted and true values, providing a direct measure of overall prediction error.
- SI: A dimensionless metric that normalises the root mean squared error by the magnitude of observed values, reflecting the relative size of prediction errors.
- CC: Quantifies the linear correlation between predicted and true values. Values closer to 1 indicate a stronger positive correlation.
- U95: Estimates the 95% uncertainty interval, indicating the overall spread and uncertainty of predictions relative to the true values.
- Reliability Percentage: Reflects the percentage of predictions whose relative average error is below a defined threshold, indicating the proportion of "acceptable" predictions.

The mathematical expressions for these metrics are given as follows:

$$RMSE = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2} \tag{17}$$

$$MAE = \frac{1}{T}\sum_{t=1}^{T}\left|y_t - \hat{y}_t\right| \tag{18}$$

$$SI = \sqrt{\frac{\sum_{t=1}^{T}\left[(y_t - \bar{y}_t) - (\hat{y}_t - \bar{\hat{y}}_t)\right]^2}{\sum_{t=1}^{T} y_t^2}} \tag{19}$$

$$CC = \frac{\sum_{t=1}^{T}\left[(y_t - \bar{y}_t)(\hat{y}_t - \bar{\hat{y}}_t)\right]}{\sqrt{\sum_{t=1}^{T}(y_t - \bar{y}_t)^2 \sum_{t=1}^{T}(\hat{y}_t - \bar{\hat{y}}_t)^2}} \tag{20}$$

$$U95 = \left(\frac{1.96}{T}\right)\sqrt{\sum_{t=1}^{T}(y_t - \bar{y}_t)^2 + \sum_{t=1}^{T}(y_t - \hat{y}_t)^2} \tag{21}$$

$$\text{Reliability} = \left(\frac{100\%}{T}\right)\sum_{t=1}^{T} J_t \tag{22}$$

here y_t and $\hat{y}_t$ denote the true and predicted values at sample t, respectively; $\bar{y}$ and $\bar{\hat{y}}$ are the mean values of y_t and $\hat{y}_t$, and T is the total number of samples.

For the reliability calculation, J_t is determined as follows:

$$RAE_t = \left|\frac{y_t - \hat{y}_t}{\hat{y}_t}\right| \tag{23}$$

where $J_t = 1$ if $RAE_t \leq \Delta$, and $J_t = 0$ otherwise. The threshold Δ defines the acceptable level of prediction error, set to 0.2 in this study according to Chinese standards.

Data introduction

Hardware and data collection

The data applied in this study was collected using a NovAtel's PwrPak7D-E1 receiver. This device combines a dual-antenna GNSS receiver (model: OEM7720) with a MEMS IMU (model: Epson G320N) through the SPAN technique to enhance navigation accuracy. The Epson G320N IMU offers a dynamic range of 150°/s for the gyroscope and 5 g for the accelerometer. The gyroscope's bias instability measures 3.5°/hr, and the accelerometer's is 0.1 mg. Moreover, the gyroscope exhibits a random walk of 0.1 °/$\sqrt{\text{hr}}$, and the accelerometer 0.05 m/s/$\sqrt{\text{hr}}$, affirming their navigational precision.

The system was installed on the vehicle as follows. A Taoglas antenna configured to track GPS and GLONASS signals on L1 and L2 frequencies was mounted on the vehicle's roof. The PwrPak7D-E1 receiver was positioned directly below the GNSS antenna inside the vehicle, adhering to the configuration instructions provided in the PwrPak7 user manual (NovAtel Inc. 2023).

Two datasets were collected for use as training and test datasets respectively. As shown in Fig. 3a. The trajectory used for collecting the training dataset is marked in red, while the trajectory for the test dataset is marked in yellow. For the training dataset, the vehicle began its journey in Ipswich city, travelling through several roads in this area before proceeding to the Bundamba area. In the Bundamba area, the vehicle navigated a variety of driving conditions, including roundabouts, performing U-turns, and making sharp turns, as depicted in Fig. 3c. The test dataset was collected within Ipswich city. Figure 3b demonstrates that to compile a comprehensive dataset, the vehicle made multiple passes through various intersections, covering all possible lanes. It is important to highlight that while both datasets were collected in Ipswich city, they followed different routes, as depicted in Fig. 3b–the test route does not completely overlap with the training route. This variance is critical for evaluating how ML models perform on unfamiliar routes. The entire journey lasted 120 min and covered 40 km,

Fig. 3 The vehicle trajectory during data collection: **a** the overall trajectory; **b** trajectory in the Ipswich city; **c** trajectory in the Bundamba area

resulting in an average speed of 20 km/h. The low average speed was primarily due to the urban traffic conditions, including delays at traffic lights and congestion.

Data format and pre-processing

The PwrPak7D-E1 was configured to provide 10 Hz GNSS/INS DCI solutions and 125 Hz IMU raw data. The GNSS/INS DCI solutions served as the true values for the vehicle's position. In addition, 10 Hz RTK position solutions were also obtained using the CLEV4 site as a reference station, connected via a 4 G network to provide the necessary corrections across a baseline distance of 17 km to 25 km.

The vehicle's OBD-II system can export speed data at a frequency of 10 Hz (Shaw et al. 2019). However, OBD-II speeds were not collected during this experiment. It is established that OBD-II-derived speeds exhibit a bounded error once the scale factor is determined (Wahlstrom et al. 2018). Therefore, this study simulated OBD-II speeds by applying a 5% scale error and 2 km/h noise to DCI-derived speeds, and this simulated data was then used in the proposed method.

In this study, the duration of GNSS outages was limited to 120 s. As a result, the datasets were segmented into multiple 120-s intervals. Within each segment, the position recorded at the first epoch served as the reference point for calculating the north and east coordinates. To evaluate the accuracy of ML-derived positions, they were compared with the positions updated by the KF method.

Results and analysis

ML training and prediction

Two ML methods, the GBDT and LSTM, were employed to model positions in N and E directions. The training datasets were normalised prior to training, with 80% utilised for model training and the remaining 20% used for validation.

A comprehensive grid search was conducted to identify optimal hyperparameter configurations for both the GBDT and LSTM models. For the GBDT model, the hyperparameters tuned included the number of trees, learning rate, maximum tree depth, minimum samples required to split, minimum samples per leaf, validation fraction, and early-stopping patience as outlined in Table 1. For the LSTM model, hyperparameters such as network architecture (number of layers and hidden units), learning rate, L2 regularisation factor, mini-batch size, optimiser type, and early-stopping patience were considered, with grid search options detailed in Table 2.

Table 1 Grid search options for key GBDT hyperparameters

Hyperparameter	Options
Number of trees	50, 100, 200, 300
Learning rate	0.01, 0.05, 0.1, 0.2
Maximum tree depth	2, 3, 4, 5, 6
Minimum samples to split	2, 5, 10
Minimum samples per leaf	1, 2, 4, 8
Validation fraction	0.1, 0.2, 0.3
Early-stopping patience	10, 40, 80

Table 2 Grid search options for key LSTM hyperparameters

Hyperparameter	Options
NN structure	1-64-32-1, 1-32-16-1, 1-64-1, 1-32-1
Learning rate	0.001, 0.005, 0.01, 0.02
L2 regularisation	0, 0.001, 0.01, 0.1
Mini-batch size	32, 64
Optimiser	Adam, rmsprop
Early-stopping patience	10, 40, 80

The optimal GBDT configuration determined by grid search included 100 estimators, a learning rate of 0.1, a maximum tree depth of 3, minimum samples to split set to 2, minimum samples per leaf set to 1, a validation fraction of 0.2, and a fixed random seed. Early stopping with a patience of 40 epochs was employed to prevent overfitting. These hyperparameter settings provided a balance between model complexity and generalisation.

For the LSTM model, the selected configuration featured a single hidden layer with 32 hidden units, trained using the 'adam' optimiser and an L2 regularisation factor of 0.01. The chosen learning rate was 0.01, and the mini-batch size was set to 32. Early stopping was utilised with a patience parameter of 40 epochs. Model performance was continuously monitored using the validation dataset.

Figure 4 shows the LSTM training progress with RMSE curves for training and validation sets in both north (N) and east (E) directions. RMSE dropped rapidly at first and then stabilised, with convergence at epoch 952 and 1336 in N and E directions, respectively. The validation RMSE closely followed the training RMSE throughout, indicating strong generalisation and no overfitting.

The training dataset comprises 1 h of data, totalling 36,000 samples. The trained models were used to predict positions during 30 additional 2-minute outages. The ML-derived positions were then compared with the true values (GNSS/INS DCI results) and the INS KF results. The results will be displayed and analysed in the following sections.

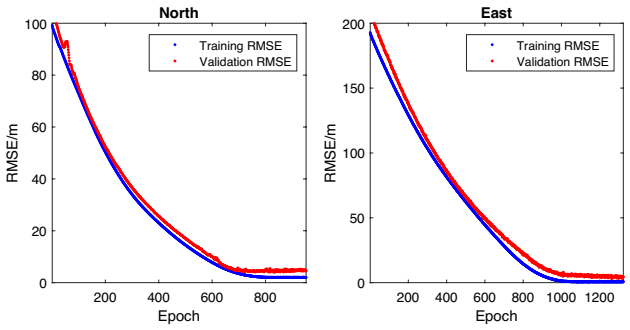

Fig. 4 LSTM learning curves in north and east directions

Performance analysis of ML models across 30 outages

The LSTM and GBDT models were tested across 30 different outages. The accuracy of the positioning results derived from these ML models, along with the KF, in the N, E, and overall horizontal directions is illustrated in Fig. 5.

In the N and E directions, both ML methods generally exhibit superior positioning accuracy compared to the KF method during most outages. However, there are rare instances where the KF achieves high accuracy, during which the ML methods may reduce positioning accuracy, as demonstrated by the results of outage 30 in N direction, and outages 3 and 4 in E direction.

Table 3 compares the performance of KF, LSTM, and GBDT in the N and E directions. Both ML methods provide a substantial reduction in positioning error compared to KF. LSTM decreases RMSE from 26.3 m (KF) to 3.8 m in N, and from 29.4 m (KF) to 4.1 m in E. GBDT also shows marked improvements, reducing RMSE to 4.5 m in N and 7.6 m in E. For horizontal accuracy, calculated via

$$\mathrm{RMSE}_{\mathrm{horizontal}} = \sqrt{(\mathrm{RMSE}_N)^2 + (\mathrm{RMSE}_E)^2}, \tag{24}$$

LSTM achieves an RMSE of 5.6 m and GBDT achieves 8.8 m, both significantly lower than the 39.5 m of KF. These results correspond to improvement rates of 85.8% for LSTM and 77.7% for GBDT over KF, demonstrating the clear advantage of ML approaches in reducing horizontal position errors.

For other evaluation metrics, including MAE, U95, SI, CC, and reliability, LSTM and GBDT also outperform KF. Both ML models achieve lower MAE and narrower U95 intervals, indicating more consistent and reliable predictions. Their SI values are much lower, reflecting reduced dispersion in errors, and CC values approach 1.00, indicating

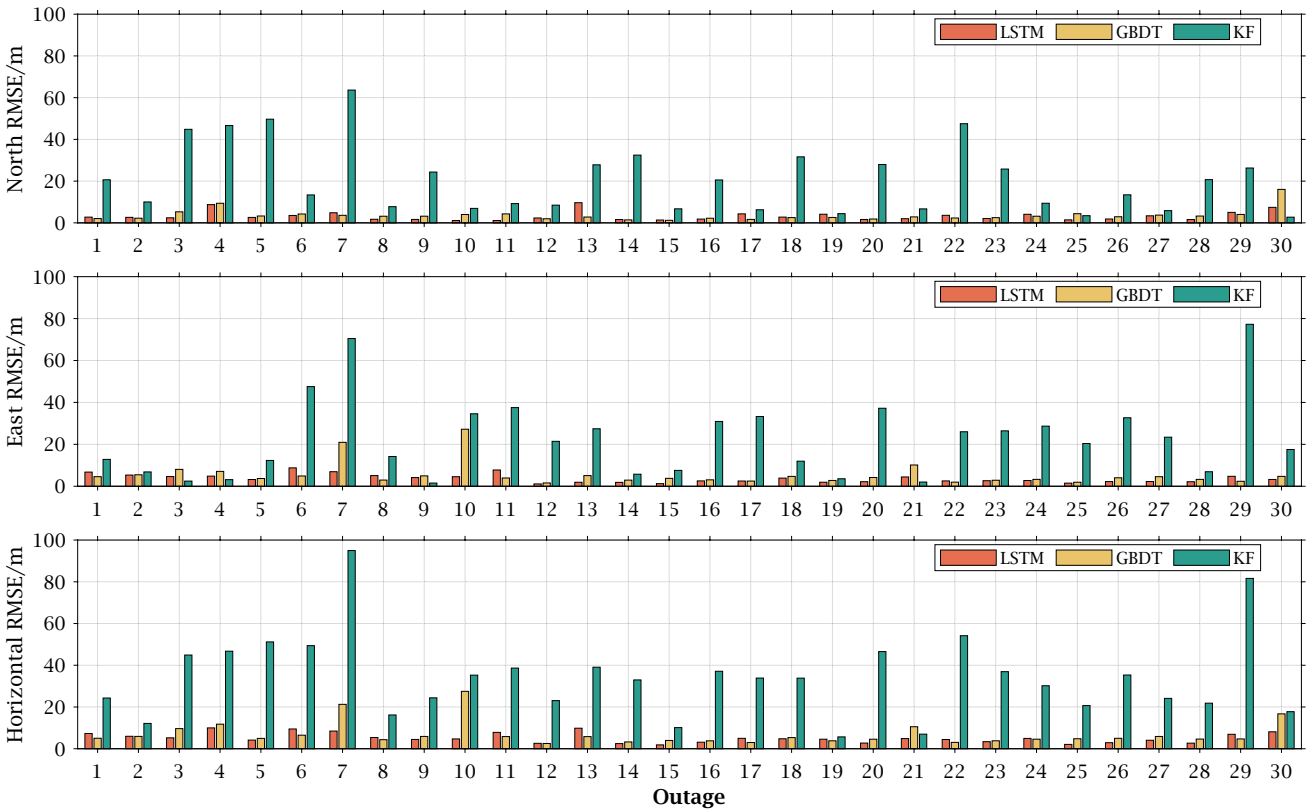

Fig. 5 Positioning accuracy for LSTM-, GBDT- and KF-derived results in N, E, and overall horizontal directions during 30 outages

strong agreement with true positions. Reliability is also notably higher for LSTM (75.8% in N, 81.7% in E) and GBDT (75.1% in N, 76.7% in E) compared to KF (49.0% in N, 63.4% in E), further supporting the superiority of ML-based methods in practical GNSS/INS positioning scenarios.

The accuracy improvement rates achieved by the ML methods compared to the KF for each outage are depicted in Fig. 6. This figure reveals that LSTM achieves more than a 50% improvement rate in almost all outages and surpasses an 80% improvement rate in 21 out of 30 (70.0%) outages. In comparison, the GBDT method, while slightly less effective, still shows an improvement rate of over 80% in 15 of 30 (50%) outages.

The statistical results show that the LSTM outperforms the GBDT in position prediction in this study. However, training the LSTM model on the 60-minute dataset using an RTX 3080Ti GPU required 28 min. In contrast, training a GBDT model from the same datasets on an i7-12700K CPU took only 2 min, highlighting the greater efficiency of the GBDT method in model training.

Results analysis for four representative outages

To provide an intuitive understanding of the performance of ML methods, the positioning errors for the KF and ML results in the north and east directions during outages 7, 26, 29, and 21 are depicted in Figs. 8 and 9, respectively. These outages were chosen for analysis due to the vehicle exhibiting varied trajectories and behaviours. The figures not only showcase outages with favourable results but also include an underperforming one (outage 21), offering a comprehensive opportunity to assess the effectiveness of the ML methods across different scenarios.

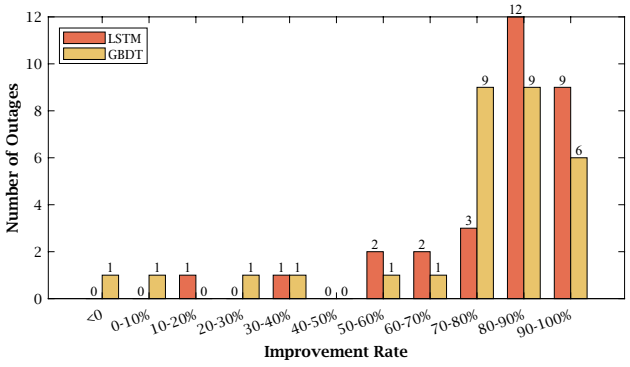

Fig. 6 Distribution of outage numbers across various accuracy improvement rate ranges

As illustrated in Fig. 10, during outage 7, the vehicle followed essentially a straight path. In contrast, during outages 21 and 29, the vehicle executed 2 and 3 turns, respectively, leading to more complex trajectories. Furthermore, during outage 26, the vehicle executed 7 turns, resulting in the most complex trace. The vehicle's speed was limited to under 60 km/h due to urban speed limits, as shown in Fig. 7. This figure also demonstrates that the vehicle frequently stopped and started during outages 7, 29, and 21, indicating a pattern influenced by urban traffic conditions.

It can be observed from Figs. 8 and 9 that the positioning errors in the KF-derived results accumulate over time. In contrast, the results from the ML methods do not exhibit this error accumulation. Although both LSTM and GBDT achieve favourable outcomes, they display distinct error characteristics. The errors in the LSTM results are smoother and more stable compared to those from GBDT, suggesting that LSTM is capable of delivering more consistent and accurate positioning. Conversely, the results from GBDT are noticeably noisier. From Fig. 10, it is evident that the trajectories derived from the ML models consistently align well with the reference trajectories. In contrast, the trajectories derived from KF tend to drift away from the true trajectories, intuitively demonstrating the superior performance of ML methods.

The LSTM models somehow also show drawbacks. In the initial epochs, the LSTM models typically cannot produce good results, which is particularly evident in Figs. 10c and d.

In addition to the study of positioning performance, the velocity prediction accuracy is also evaluated. Figure 11 illustrates the velocity prediction errors of KF, GBDT, and LSTM during those four representative outages. In both ML methods, velocity was not directly estimated as an output; instead, it was calculated from the predicted positions. As demonstrated in Fig. 11, LSTM consistently delivers stable and minimal velocity errors across all scenarios. Its predictions remain close to zero, showing

Table 3 Performance metrics for North and East directions using KF, LSTM, and GBDT

Direction	Method	RMSE (m)	MAE (m)	U95 (m)	SI	CC	Reliability (%)
North	KF	26.3	14.8	3.6	0.24	0.94	49.0
	LSTM	3.8	2.4	0.5	0.03	1.00	75.8
	GBDT	4.5	2.8	0.6	0.04	1.00	75.1
East	KF	29.4	16.2	3.9	0.13	0.96	63.4
	LSTM	4.1	2.4	0.6	0.02	1.00	81.7
	GBDT	7.6	3.8	1.1	0.03	1.00	76.7

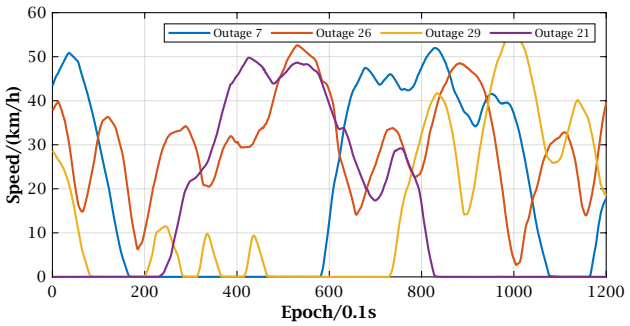

Fig. 7 Vehicle speeds during four representative GNSS signal outages

resilience to frequent stops, turns, and speed fluctuations. In contrast, GBDT, despite outperforming KF, exhibits significant noise and fluctuations. The KF approach demonstrates a clear trend of accumulating velocity errors over time, notably evident in outages 26 and 29, reflecting difficulties in accurately predicting velocity during prolonged GNSS outages.

These observations indicate that, besides positioning, LSTM provides superior velocity estimation reliability under diverse driving conditions, while GBDT, though advantageous over KF, struggles with abrupt trajectory dynamics. KF consistently shows the highest susceptibility to cumulative errors during extended outages.

ML performance during different periods of each outage

This subsection examines LSTM and KF performance across different time intervals during the outage. The 120-second outages were divided into 12 intervals. The positioning accuracy of both LSTM and KF methods was investigated across these intervals. Figure 12 displays the average positioning accuracy for KF and LSTM during each interval among all 30 studied outages. This figure indicates that KF accuracy decreases rapidly over time. In contrast, LSTM accuracy demonstrates a significantly slower rate of accuracy degradation. In the first 10 s, KF shows higher accuracy than LSTM, with respective accuracies of 0.4 and 7.1 m. During the second interval (11–20 s), both methods achieved similar positioning accuracies, approximately 1.3 m. After this interval, LSTM consistently shows higher accuracy than KF, with the accuracy difference widening due to varying rates of degradation. During the third interval, KF and LSTM accuracies were 3.2 and 1.9 m respectively. In the last interval, KF accuracy plummeted to 85.7 m while LSTM maintained an accuracy of 9.4 m, reflecting a 89.0% improvement. These results suggest that the LSTM method offers more advantages in maintaining accuracy over longer outage durations compared to the traditional KF method.

Study on the robustness of ML method

In this paper, the definition of robustness for the ML method includes two aspects: (1) How errors in the outputs of the

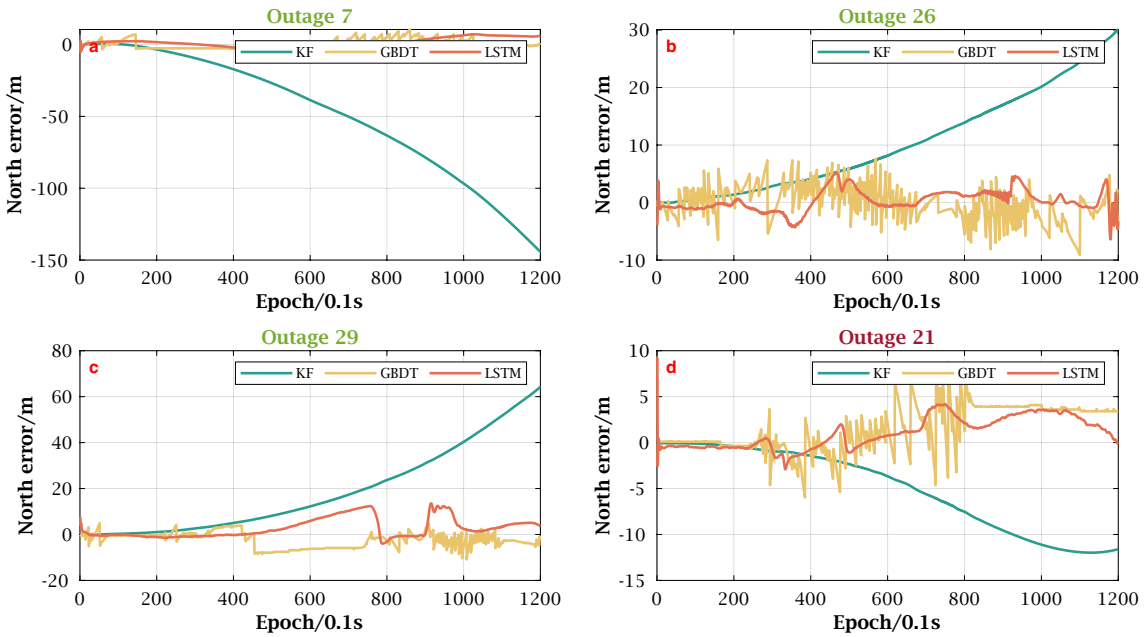

Fig. 8 Positioning errors for KF, GBDT and LSTM results in N direction during four representative GNSS signal outages

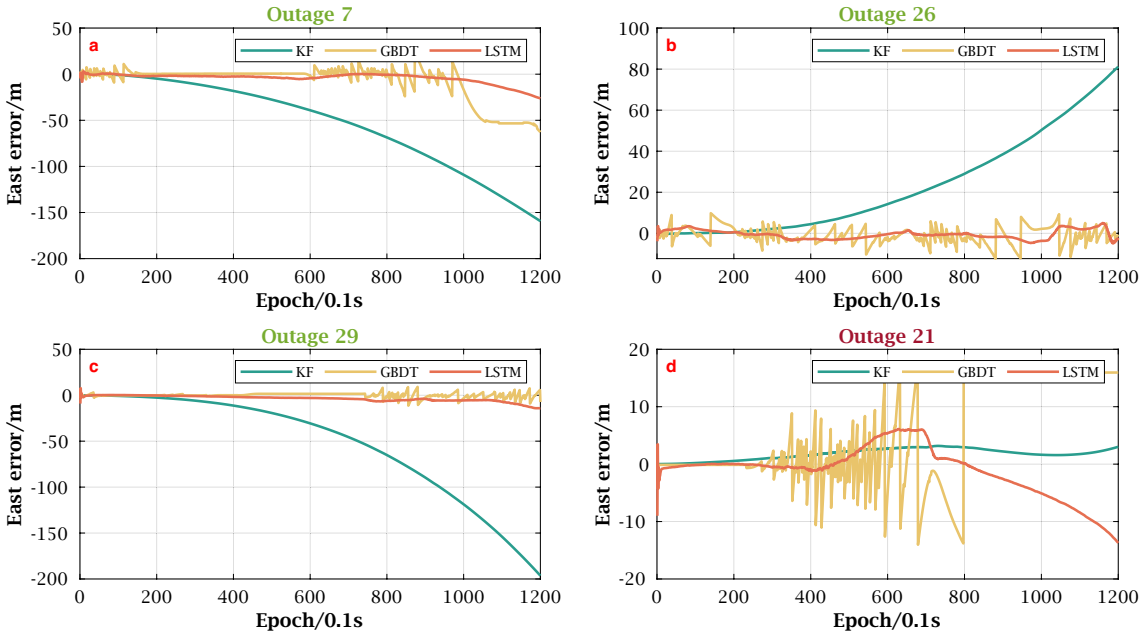

Fig. 9 Positioning errors for KF, GBDT and LSTM results in E direction during four representative GNSS signal outages

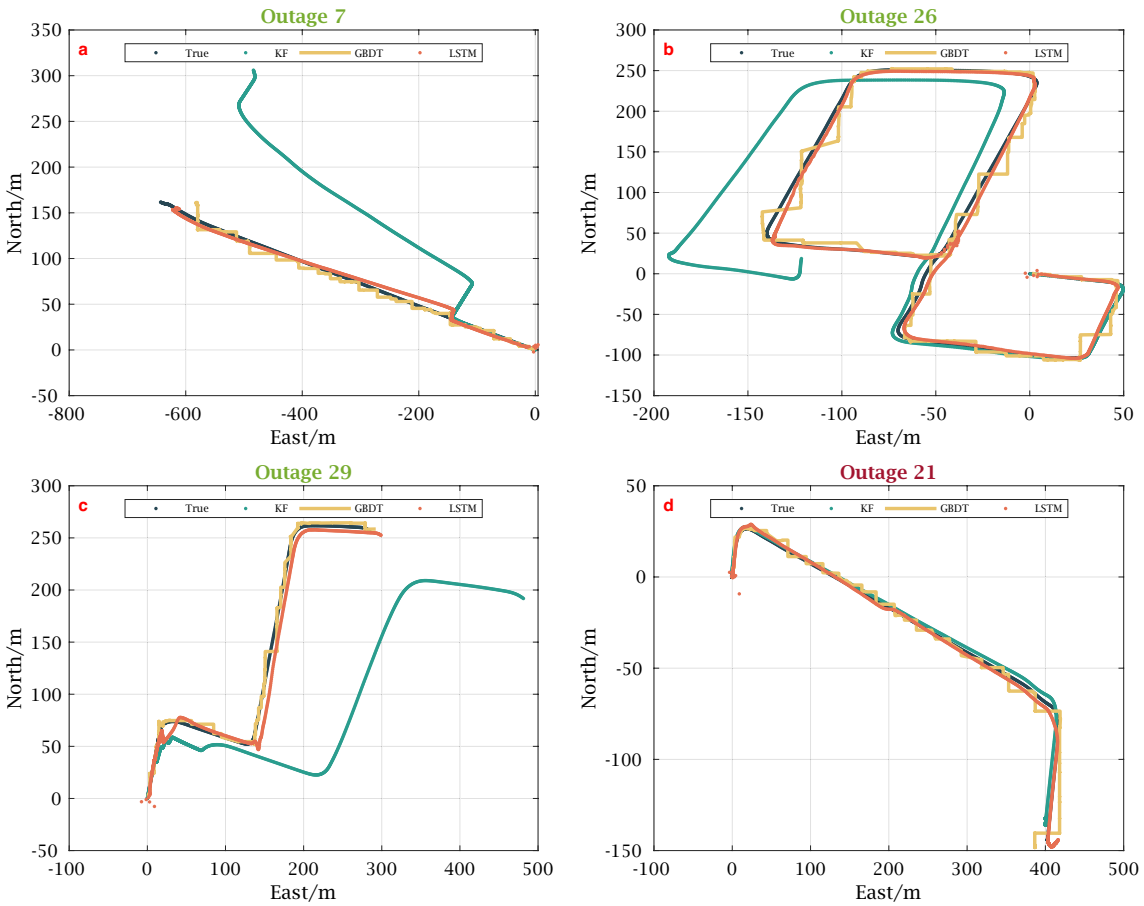

Fig. 10 The true, KF-, GBDT- and LSTM-derived vehicle tracks during four representative GNSS signal outages

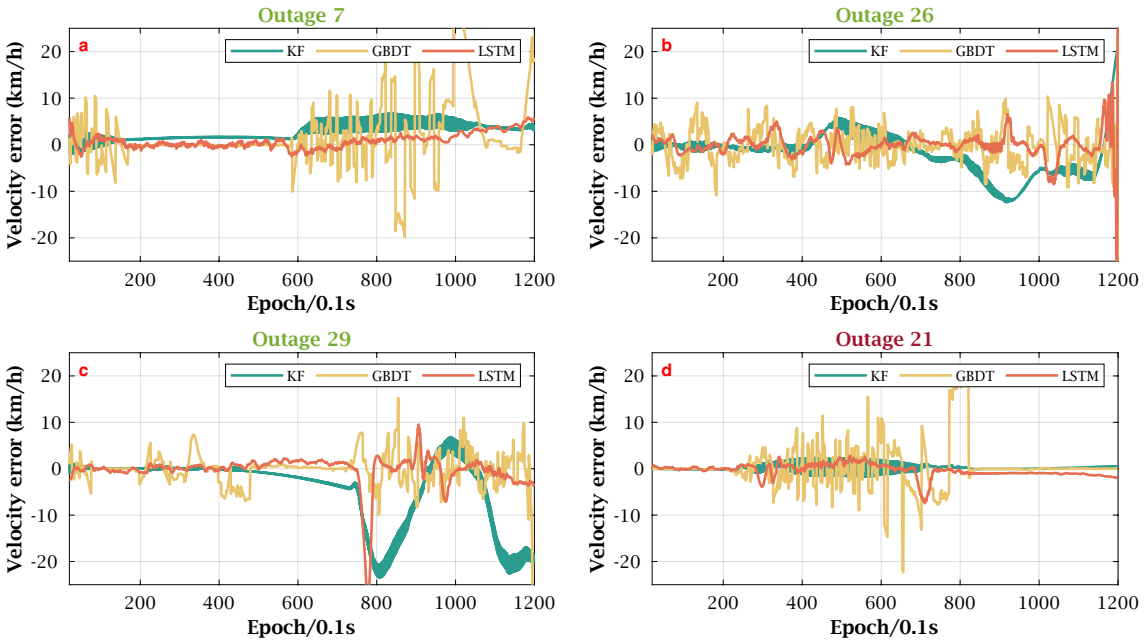

Fig. 11 Velocity errors for KF, GBDT and LSTM results

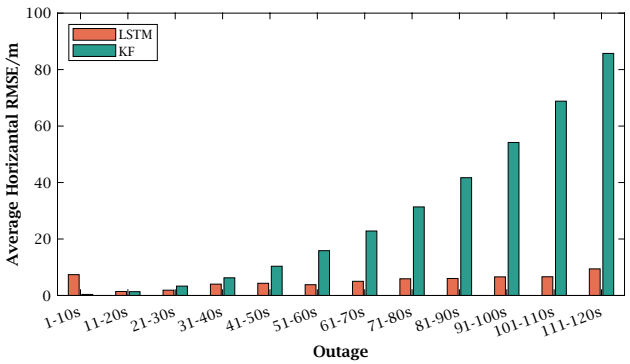

Fig. 12 Average horizontal positioning accuracy across different time intervals

training dataset affect the quality of the model, and (2) How the prediction performance varies when the input variables exhibit higher error levels, including noise and drift. This section will explore these concerns.

Impact of Y errors in training dataset on the model performance

In previous experiments, positions derived from GNSS/INS DCI results were used as outputs for ML training. This integration is beneficial, yielding smooth, low-noise positioning results that enhance ML training. However, this technology is proprietary to companies that have developed their own RTK chips, which limits the broader application of our proposed method. When the DCI results are unavailable, RTK positioning results can serve as an alternative. RTK results, in comparison to DCI results, are noisier and prone to outliers due to poor geometry, signal loss, or multipath effects. Additionally, when using lower-grade GNSS receivers or the DGPS technique, positioning results exhibit increased white noise. To evaluate how these outliers and noise impact ML training, this section discusses the outcomes from three ML models trained using different RTK-based positioning data. These models are referred to as RTK-1, RTK-2, and RTK-3. RTK-1 utilised unprocessed RTK positioning data for ML training. RTK-2 involved training after removing outliers from the raw RTK data. When training RTK-3, 2-meter noise was added to the cleaned RTK data used in RTK-2. Figure 13 illustrates the differences between the positioning data used in the training of three models and the DCI results (considered as true values). Figure 13a clearly shows the presence of outliers in the RTK data. After removing these anomalies, as shown in Fig. 13b, the discrepancy between the RTK and true results is reduced to less than 0.5 m. Subsequently, Fig. 13c shows the impact of adding noise (STD = 2 m) to the data.

These RTK-based positioning data were employed to train LSTM models under the same settings as the initial experiment. The derived models were then used to predict positions during 30 outages, and the statistical results are presented in Table 4. This table reveals that model RTK-1 achieved slightly lower positioning accuracy compared to the model based on DCI results, indicating that using RTK data as training outputs marginally reduces positioning accuracy during the tested outages. However, the LSTM model

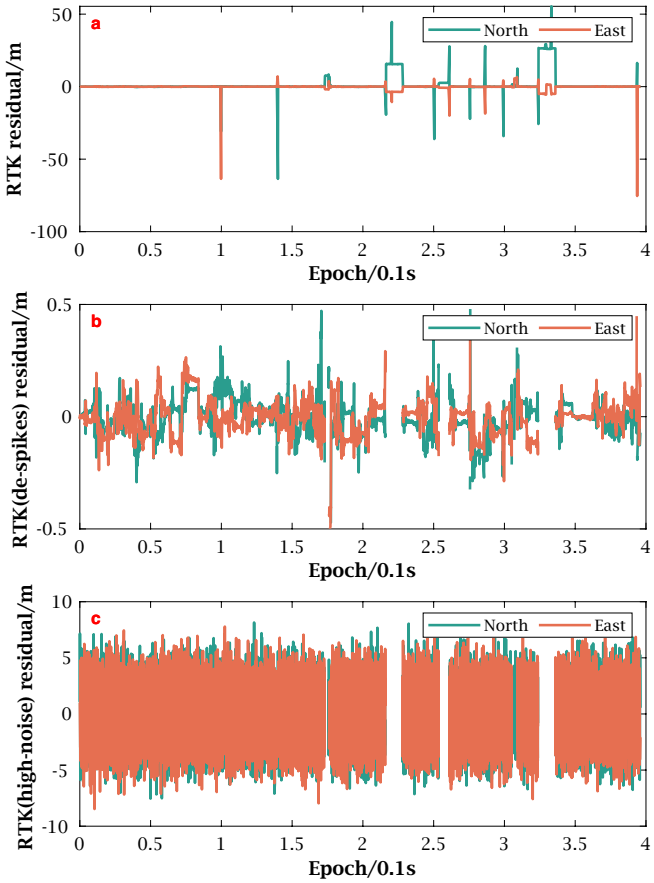

Fig. 13 Position residuals between DCI and RTK based techniques

Table 4 Positioning accuracy comparison among KF method and LSTM models trained from DCI and various RTK-based results in the N, E, and overall horizontal directions (Unit: m)

	KF	DCI	RTK-1	RTK-2	RTK-3
N	26.3	3.8	4.2	4.1	4.4
E	29.4	4.1	4.3	4.2	4.3
Horizontal	39.5	5.6	6.0	5.9	6.2

trained with RTK-derived outputs (model RTK-1) still achieved respectable position prediction performance-−6.0 m horizontally. After removing outliers from the RTK data, the corresponding model (RTK-2) achieved a positioning accuracy of 5.9 m horizontally. After adding noise to the RTK data, the subsequent model (RTK-3) achieved a positioning accuracy of 6.2 m horizontally. The comparison of the results derived from models RTK-1, RTK-2, and RTK-3 demonstrates that the proposed LSTM processing strategy is robust against outliers and noise in response data.

Impact of X errors in testing dataset on prediction

The ML models previously trained exhibited high prediction accuracy when tested under conditions similar to the training dataset. However, in some scenarios, the input variables in the testing dataset could display different error patterns, such as when the dataset collected from the IMU differs from that used during collection of training data. This part will discuss how errors in input variables affect prediction accuracy.

In this study, four input variables were used: elapsed time (Input A), IMU-derived position (Input B), OBD-derived position (Input C), and azimuth (Input D). To simplify the experiment, only the input variable with the most significant impact on prediction outcomes will be studied. Figure 14 displays the permutation feature importance (Fisher et al. 2019) of the four input variables for two DCI data-derived LSTM models (in the N and E directions). The feature importance rankings were consistent across both models: Input C was the most critical, followed by A, B, and D. Based on these results, Input C's impact on prediction accuracy will be further explored.

The accuracy of azimuth is pivotal as it influences three of the four input variables. We therefore assume the use of an IMU equipped with a lower-grade gyro for collecting test data. The azimuth derived from this gyro will be simulated by introducing noise and drift to the raw azimuth dataset.

The simulated azimuth datasets were generated as follows:

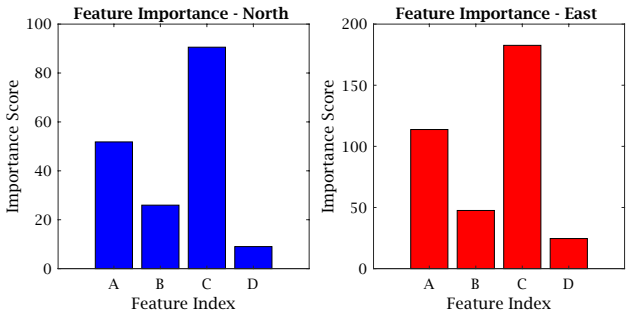

Fig. 14 Permutation feature importance of the four input variables for DCI data-derived LSTM models in N and E directions

$$a_s^{(i,j)} = a_{INS} + U^{(i)} + D^{(j)} \tag{25}$$

where $i = 1, 2, \ldots, 5$ and $j = 1, 2, \ldots, 5$. Here, $U^{(i)}$ represents the noise with a STD of $5i$ °/h, and $D^{(j)}$ represents the azimuth drift at a linear rate of $0.2j$ °.

The OBD-derived position will then be updated based on these simulated azimuth datasets. The updated OBD-derived position, along with the other three input variables, were used to predict positions, and the prediction accuracy was analysed. The horizontal prediction RMSE for the $ij = 25$ sets of results is illustrated in Fig. 15. The figure demonstrates that the ML model maintains stable prediction accuracy under various levels of azimuth noise and bias. Even when the standard deviation of the added noise reaches 1.0°, the increase in RMSE is negligible, remaining below 0.3 m. Similarly, the impact of azimuth bias–ranging from 5 to 25 °/h–is minimal, with horizontal prediction RMSE consistently falling within a narrow range of 5.5 m to 6.1 m. Overall, prediction accuracy stays around 6 m when the added bias is 25 °/h or less and the noise STD is no more than 1.0 °, showcasing the ML model's robust performance.

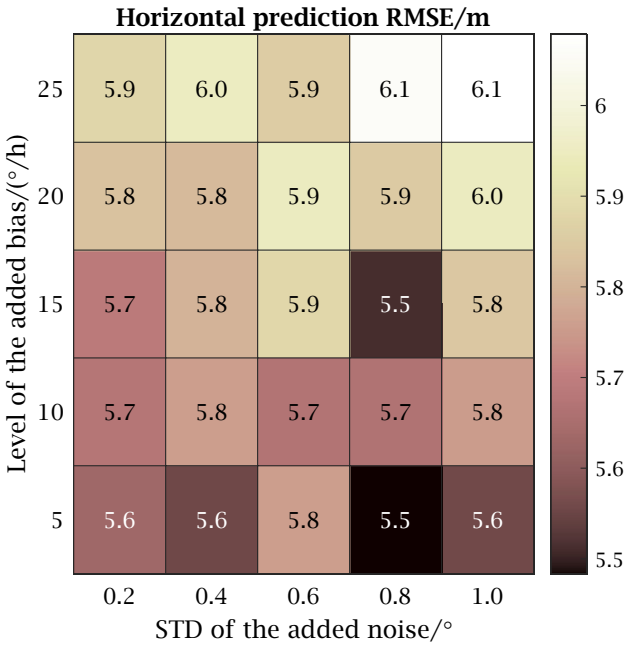

Fig. 15 ML position prediction RMSEs derived from test datasets with different levels of noises and biases

Conclusions

In summary, the research examined the robustness characteristics of ML enhanced vehicle positioning results, drawing on data from different GNSS/INS processing methods. The findings indicate the effectiveness of ML methods in improving vehicle positioning accuracy during GNSS signal outages, representing a notable advancement over KF methods. These ML strategies have demonstrated their value by markedly reducing the average positional RMSE from 39.5 m to 5.6 and 8.8 m, for LSTM and GBDT respectively, during 120-second signal outages. This equates to a substantial improvement rate of 85.8% for LSTM and 77.7% for GBDT. The results highlighted the LSTM approach's superior consistency and robustness in processing diverse datasets–such as DCI and RTK–and its resilience against meter-level noise and biases in the positioning data. Furthermore, LSTM has shown an impressive capability to withstand certain degrees of input data perturbation, such as azimuth discrepancies.

The significance of the findings lies in the practical applicability of ML approaches in the road environments. For instance, road navigation can use the ML predicted solution to bridge vehicle navigation outages when travelling restricted roads. The connected and automated vehicles may use the predicted vehicle location knowledge for vehicle collision avoidance computation. Nevertheless, it's worth acknowledging the limitations of our study. We did not account for high-velocity vehicle travel conditions in our research model, and, as such, the robustness of ML strategies at elevated speeds remains untested. Additionally, the vehicle speed data, rather than being sourced directly from real-world OBD-II systems, was simulated with added uncertainty. To fortify the reliability of these ML techniques, future research endeavours should integrate data from actual OBD-II systems to further affirm the robustness and applicability of ML for vehicular positioning. This step would help to validate and potentially refine the effectiveness of these ML approaches under a broader and more dynamic range of real-world driving conditions. Despite these caveats, the advancements showcased in this study pave the way for integrating ML into modern vehicular systems, with the promise of significantly enhanced navigational accuracy and reliability.

Acknowledgements The first author acknowledges the financial support received from the Queensland University of Technology and China Scholarship Council. The authors would like to acknowledge the team of Prof. Xiaoji Niu of the Integrated and Intelligent Navigation (i2Nav) group from GNSS Research Center of Wuhan University for providing the open-source KF-GINS software that was used in the paper.

Author contributions W.G. prepared the data, created the processing codes, performed the analysis and drafted the paper. Y.F. initiated the topic, conceived and designed the analysis and improved the paper. All authors read and approved the final manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Ethical approval Not applicable.

References

Abdel-Hamid W, Noureldin A, El-Sheimy N (2007) Adaptive fuzzy prediction of low-cost inertial-based positioning errors. IEEE Trans Fuzzy Syst 15(3):519–529. https://doi.org/10.1109/tfuzz.2006.889936

Adusumilli S, Bhatt D, Wang H, Bhattacharya P, Devabhaktuni V (2013) A low-cost ins/gps integration methodology based on random forest regression. Expert Syst Appl 40(11):4653–4659

Al Bitar N, Gavrilov A (2021) A novel approach for aiding unscented Kalman filter for bridging GNSS outages in integrated navigation systems. Navigation 68(3):521–539

Al Bitar N, Gavrilov A, Khalaf W (2020) Artificial intelligence based methods for accuracy improvement of integrated navigation systems during gnss signal outages: An analytical overview. Gyroscopy Navig 11:41–58

Amami MM (2022) The advantages and limitations of low-cost single frequency GPS/mems-based ins integration. Glob J Eng Technol Adv 10(2):018–031

Chen L, Fang J (2014) A hybrid prediction method for bridging GPS outages in high-precision pos application. IEEE Trans Instrum Meas 63(6):1656–1665

Chen X, Shen C, Wb Zhang, Tomizuka M, Xu Y, Chiu K (2013) Novel hybrid of strong tracking Kalman filter and wavelet neural network for GPS/ins during GPS outages. Measurement 46(10):3847–3854

Dasanayaka N, Feng Y (2022) Analysis of vehicle location prediction errors for safety applications in cooperative-intelligent transportation systems. IEEE Trans Intell Transp Syst 23(9):15512–15521. https://doi.org/10.1109/tits.2022.3141710

Dong Y, Fu QA, Yang X, Pang T, Su H, Xiao Z, Zhu J (2020) Benchmarking adversarial robustness on image classification. In: proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp 321–331

Ebrahimi A, Nezhadshahbodaghi M, Mosavi MR, Ayatollahi A (2024) An improved GPS/INS integration based on EKF and ai during GPS outages. J Circuits Syst Comput 33(02):2450035

El Sabbagh MS, Maher A, Abozied MA, Kamel AM (2023) Promoting navigation system efficiency during GPS outage via cascaded neural networks: a novel ai based approach. Mechatronics 94(103):026

El-Sheimy N, Chiang KW, Noureldin A (2006) The utilization of artificial neural networks for multisensor system integration in navigation and positioning instruments. IEEE Trans Instrum Meas 55(5):1606–1615. https://doi.org/10.1109/tim.2006.881033

Fang W, Jiang J, Lu S, Gong Y, Tao Y, Tang Y, Yan P, Luo H, Liu J (2020) A LSTM algorithm estimating pseudo measurements for aiding ins during GNSS signal outages. Remote Sens 12(2):256. https://doi.org/10.3390/rs12020256

Fisher A, Rudin C, Dominici F (2019) All models are wrong, but many are useful: learning a variable's importance by studying an entire class of prediction models simultaneously. J Mach Learn Res 20(177):1–81

Frénay B, Verleysen M (2013) Classification in the presence of label noise: a survey. IEEE Trans Neural Netw Learn Syst 25(5):845–869

Friedman JH (2001) Greedy function approximation: a gradient boosting machine. Ann Stat 29(5):1189–1232

Geirhos R, Jacobsen JH, Michaelis C, Zemel R, Brendel W, Bethge M, Wichmann FA (2020) Shortcut learning in deep neural networks. Nat Mach Intell 2(11):665–673

Goodall CL (2009) Improving usability of low-cost ins/gps navigation systems using intelligent techniques. Thesis

Hendrycks D, Basart S, Mu N, Kadavath S, Wang F, Dorundo E, Desai R, Zhu T, Parajuli S, Guo M et al. (2021) The many faces of robustness: a critical analysis of out-of-distribution generalization. In: Proceedings of the IEEE/CVF international conference on computer vision. pp 8340–8349

Hong S, Lee MH, Chun HH, Kwon SH, Speyer JL (2005) Observability of error states in GPS/INS integration. IEEE Trans Veh Technol 54(2):731–743

Hu G, Wang W, Zhong Y, Gao B, Gu C (2018) A new direct filtering approach to INS/GNSS integration. Aerosp Sci Technol 77:755–764

Jingsen Z, Wenjie Z, Bo H, Yali W (2016) Integrating extreme learning machine with Kalman filter to bridge GPS outages. IEEE. https://doi.org/10.1109/icisce.2016.98

Koh PW, Sagawa S, Marklund H, Xie SM, Zhang M, Balsubramani A, Hu W, Yasunaga M, Phillips RL, Gao I et al. (2021) Wilds: a benchmark of in-the-wild distribution shifts. In: International conference on machine learning. PMLR, pp 5637–5664

Kw Chiang, Noureldin A, El-Sheimy N (2008) Constructive neural-networks-based MEMS/GPS integration scheme. IEEE Trans Aerosp Electron Syst 44(2):582–594

Ma X, Tao Z, Wang Y, Yu H, Wang Y (2015) Long short-term memory neural network for traffic speed prediction using remote microwave sensor data. Transp Res Part C Emerg Technol 54:187–197

Meng Y, Gao S, Zhong Y, Hu G, Subic A (2016) Covariance matching based adaptive unscented Kalman filter for direct filtering in INS/GNSS integration. Acta Astronaut 120:171–181

Noureldin A, Osman A, El-Sheimy N (2003) A neuro-wavelet method for multi-sensor system integration for vehicular navigation. Meas Sci Technol 15(2):404

Noureldin A, El-Shafie A, Taha MR (2007) Optimizing neuro-fuzzy modules for data fusion of vehicular navigation systems using temporal cross-validation. Eng Appl Artif Intell 20(1):49–61

Noureldin A, Karamat TB, Eberts MD, El-Shafie A (2009) Performance enhancement of mems-based INS/GPS integration for low-cost navigation applications. IEEE Trans Veh Technol 58(3):1077–1096. https://doi.org/10.1109/tvt.2008.926076

NovAtel Inc (2022) SPAN overview and integration guide. https://hexagondownloads.blob.core.windows.net/public/Novatel/assets/Documents/Papers/APN-109-SPAN-Overview-and-Integration-Guide/APN-109-SPAN-Overview-and-Integration-Guide.pdf, aPN-109

NovAtel Inc (2023) PwrPak7 family installation and operation user manual. URL: chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://docs.novatel.com/OEM7/Content/PDFs/PwrPak7_Installation_Operation_Manual.pdf

Schmidt G, Barbour N, Robertson P, Hopkins III R, Angermann M, Veth M (2011) Low-cost navigation sensors and integration technology. RTO lecture series

Sharaf R, Noureldin A (2007) Sensor integration for satellite-based vehicular navigation using neural networks. IEEE Trans Neural Netw 18(2):589–594. https://doi.org/10.1109/tnn.2006.890811

Sharaf R, Noureldin A, Osman A, El-Sheimy N (2005) Online INS/GPS integration with a radial basis function neural network. IEEE Aerosp Electron Syst Mag 20(3):8–14. https://doi.org/10.1109/maes.2005.1412121

Shaw S, Hou Y, Zhong W, Sun Q, Guan T, Su L (2019) Instantaneous fuel consumption estimation using smartphones. IEEE. https://doi.org/10.1109/vtcfall.2019.8891261

Sugiyama M, Krauledat M, Müller KR (2007) Covariate shift adaptation by importance weighted cross validation. J Mach Learn Res 8(5):985–1005

Wahlstrom J, Skog I, Nordstrom RL, Handel P (2018) Fusion of OBD and GNSS measurements of speed. IEEE Trans Instrum Meas 67(7):1659–1667. https://doi.org/10.1109/tim.2018.2803998

Wang JJ, Wang J, Sinclair D, Watts L (2007) Neural network aided kalman filtering for integrated gps/ins geo-referencing platform. In: Proc. 5th Int. Symp. Mobile Mapping Technol. pp 1–6

Wang G, Xu X, Yao Y, Tong J (2019) A novel bpnn-based method to overcome the GPS outages for INS/GPS system. IEEE Access 7:82134–82143

Wang X, Chen JX, Ni W (2015) A hybrid prediction method and its application in the distributed low-cost INS/GPS integrated navigation system. In: 2015 18th international conference on information fusion (Fusion). IEEE, pp 1205–1212

Yang L, Li Y, Wu Y, Rizos C (2014) An enhanced mems-INS/GNSS integrated system with fault detection and exclusion capability for land vehicle navigation in urban areas. GPS Solut 18:593–603

Yao Y, Xu X, Zhu C, Chan CY (2017) A hybrid fusion algorithm for GPS/ins integration during GPS outages. Measurement 103:42–51

Yigit CO, Gikas V, Alcay S, Ceylan A (2014) Performance evaluation of short to long term GPS, Glonass and GPS/GLONASS post-processed PPP. Surv Rev 46(336):155–166

Zhao X, Dou L, Su Z, Liu N (2018) Study of the navigation method for a snake robot based on the kinematics model with mems IMU. Sensors 18(3):879